

App Development Project Report

Placement Portal Application (PPA)

1. Student Details

Name: Swati Singh

Roll Number: 23f3000333

Email: 23f3000333@ds.study.iitm.ac.in

About : I am Swati Singh pursuing this degree standalone currently in diploma level and passionate about web development .

2. Project Details

Project Title

Placement Portal Application (PPA)

Problem Statement

To design and build a full-stack web application that streamlines the campus placement process for students, companies, and institutions. The application allows companies to post placement drives, students to apply and track their status, and admins to manage everything from a central dashboard.

Approach

The app was built using Flask as the backend FLASK API framework and Vue.js as the frontend . Redis is used for caching and as the Celery message broker. Celery with Celery Beat handles background jobs including daily email reminders and monthly activity reports. The frontend communicates with the backend entirely via Axios API calls.

3. AI / LLM Declaration

No AI or LLM tools were used in implementing core functionality.

4. Technologies and Frameworks Used

- **Flask** - Core backend web framework for REST API development
- **SQLAlchemy** - ORM for SQLite database interaction
- **SQLite** - Lightweight relational database for storing all application data
- **Vue.js 3** - Frontend SPA framework for building reactive user interfaces
- **Bootstrap 5** - Frontend styling and responsive design
- **Flask-Mail** - Sending email notifications, reminders, and reports
- **Celery** - Asynchronous task queue for background job execution
- **Celery Beat** - Periodic task scheduler for daily and monthly scheduled jobs
- **Redis** - Message broker for Celery and caching layer for API responses
- **Flask-Caching** - Redis-backed caching for dashboard API responses
- **Vue Router** - Client-side routing with role-based route guards
- **Axios** - HTTP client for API communication between frontend and backend

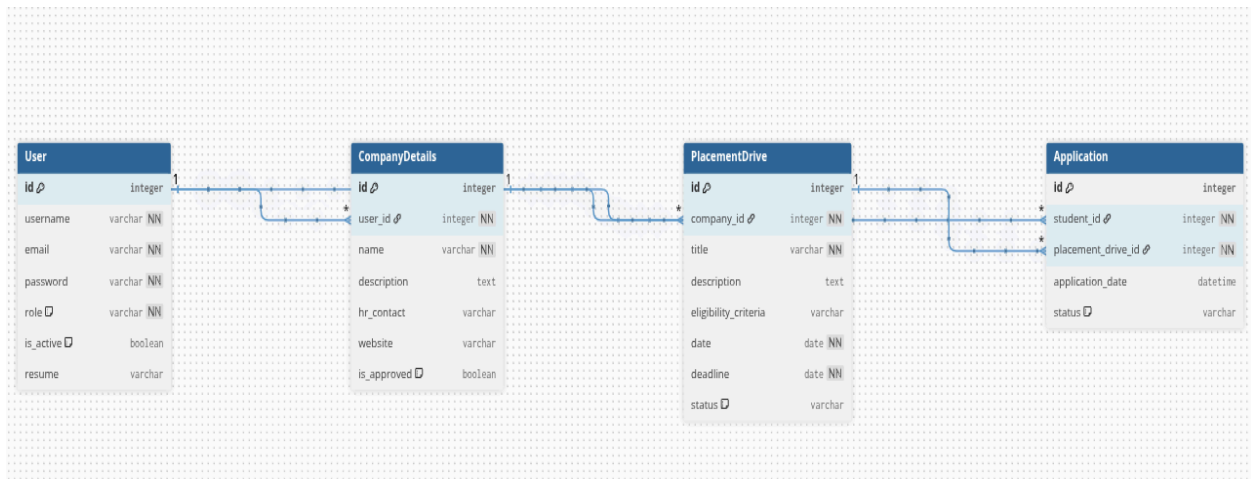
5. Database Schema Design

Tables:

1. **User** - stores all user details (id, username, email, password, role, is_active, resume)
2. **CompanyDetails** - stores company profile linked to a User (id, user_id, name, description, hr_contact, website, is_approved)
3. **PlacementDrive** - stores placement drives posted by companies (id, company_id, title, description, eligibility_criteria, date, deadline, status)
4. **Application** - stores student applications for drives (id, student_id, placement_drive_id, application_date, status)

Relationships:

- One-to-One → User → CompanyDetails
- One-to-Many → CompanyDetails → PlacementDrive
- One-to-Many → User → Application
- One-to-Many → PlacementDrive → Application



6. API Resource Endpoint

- POST /api/login - Authenticate user, return role and user_id
- POST /api/register - Register new Student or Company
- GET /api/admin/dashboard_data - Fetch stats, students, companies, drives
- GET /api/admin/search - Search students, companies, or drives
- POST /api/admin/approve_company/<id> - Approve a company
- POST /api/admin/reject_company/<id> - Reject and delete a company
- POST /api/admin/approve_drive/<id> - Approve a placement drive
- POST /api/admin/reject_drive/<id> - Reject a placement drive
- POST /api/admin/toggle_user_status/<id> - Blacklist or activate a user
- GET /api/admin/all_applications - View all student applications globally
- GET /api/company/dashboard/<user_id> - Fetch company details and drives
- POST /api/company/create_drive - Create a new placement drive

- GET /api/company/applications/<drive_id> - Get applicants for a drive
- POST /api/company/update_status - Update student application status
- POST /api/company/schedule_interview — Schedule interview date for an applicant
- GET /api/student/dashboard/<user_id> - Fetch all approved drives
- GET /api/student/profile/<user_id> - Get student profile
- GET /api/student/history/<user_id> - Get application history
- POST /api/student/apply - Apply for a placement drive
- POST /api/student/update_profile - Update email and upload resume
- POST /api/student/export-history - Trigger async CSV export via Celery

7. Architecture and Features

The application follows a client-server architecture where Flask serves as the REST API backend and Vue.js 3 handles the frontend as a Single Page Application. The backend is structured using Flask Blueprints with SQLite as the database managed via SQLAlchemy. Authentication is handled using JWT tokens. Redis is used for caching API responses and as a message broker for Celery background jobs. The system supports three roles - Admin, Company, and Student each with dedicated dashboards and functionalities. Key features include company approval workflow, CGPA-based eligibility validation, drive management, application tracking, interview scheduling, daily email reminders, monthly admin reports, and async CSV export of application history.

8. Video Presentation

Link is here:

<https://drive.google.com/file/d/1m8AvCy-hjowJ6EPyNPmprZQaKOsvMNjl/view?usp=sharing>